实验二：搜索算法

实验代码：https://github.com/Robert4063/ContemporaryAI-26-fall lab_2分支

实验目标：实现A*、Minimax与Alpha-beta，求解最小代价路径和对抗环境中的最优决策；比较启发函数、搜索深度、访问顺序的效果，并通过自设地图和八宝藏场景检验排序与缓存优化。

统一设置：Python 3.13.2、Windows 11、随机种子20261105。计时组预热3次、测量21批，取单次耗时中位数；课程样例每批10次，拓展每批1次。计时包含校验与统计，实验三包含树构造。A*扩展数不含终点，博弈访问数含根、叶子及缓存命中。

1. 实验一 迷宫寻路

目标：在四邻接迷宫中返回从S到E的最短坐标路径，比较h=0与曼哈顿距离的搜索开销。

设计：状态为(row,col)，g为起点到当前格的最佳已知代价，h=D=|row-row_E|+|col-col_E|，f=g+h。每步费用1，因此D是剩余步数下界。用heapq取最小f，同f取较小h，再按插入顺序；new_g更小时更新g_score与前驱，旧堆记录出堆后丢弃，终点有效出堆后回溯路径。

设置：采用PDF的5×5地图，S=(0,0)、E=(3,4)，X为障碍，按上、右、下、左生成邻居。两种h共用地图、移动顺序和同分规则。

结果：两种设置均返回7步路径；无解接口返回[]。启发函数对照如下。

h	最短步数	扩展节点	入堆次数	耗时/μs
0	7	15	17	21.97
D	7	7	11	16.83

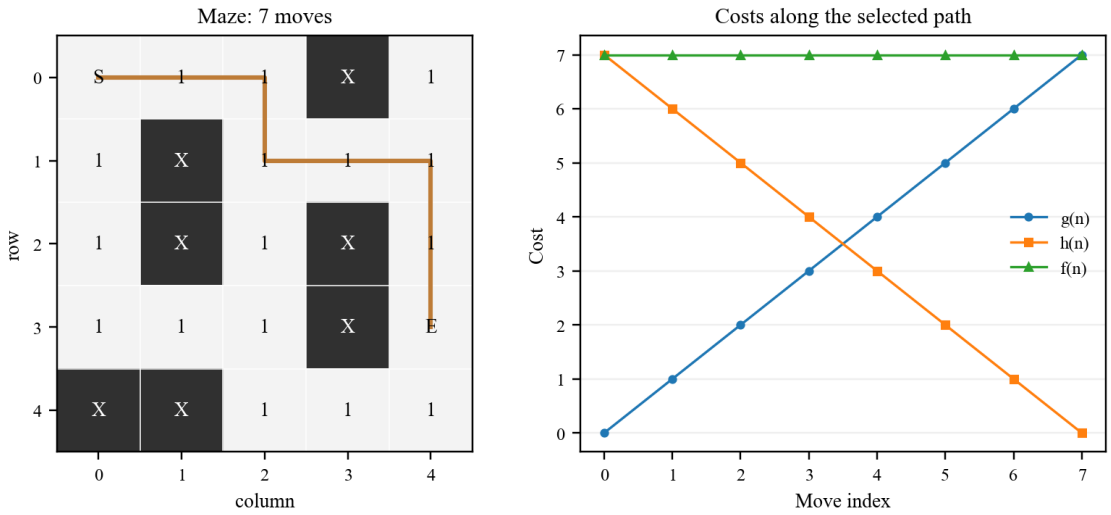


图1 左为最短路径，右为路径上各节点的g、h、f。f始终为7，g随移动增加，h同步下降。

分析：路径为(0,0)→(0,1)→(0,2)→(1,2)→(1,3)→(1,4)→(2,4)→(3,4)，达到起点曼哈顿下界7。前四个(g,h,f)为(0,7,7)、(1,6,7)、(2,5,7)、(3,4,7)。h=0按已付费用向外扩展，曼哈顿下界优先保留目标方向的候选，同f时较小h继续推动搜索前进，扩展数由15降至7，减少53.3%。

下表列出全部有效出堆记录：h=0的前、后8次分列；最后一次为终点出堆。位置后的数字分别是g或(g,h,f)。

行号	h=0 第1-8次：位置；g	h=0 第9-16次：位置；g	h=D 第1-8次：位置；(g,h,f)
1	(0,0)；0	(2,2)；4	(0,0)；(0,7,7)
2	(0,1)；1	(3,1)；4	(0,1)；(1,6,7)
3	(1,0)；1	(1,4)；5	(0,2)；(2,5,7)
4	(0,2)；2	(3,2)；5	(1,2)；(3,4,7)
5	(2,0)；2	(0,4)；6	(1,3)；(4,3,7)
6	(1,2)；3	(2,4)；6	(1,4)；(5,2,7)
7	(3,0)；3	(4,2)；6	(2,4)；(6,1,7)
8	(1,3)；4	(3,4)；7	(3,4)；(7,0,7)

总结：单位移动代价下，A*找到了达到距离下界的最短路径；加入目标距离信息后，扩展节点与实测耗时同时减少。

2. 实验二 加权地形

目标：在不同进入费用的地形上求最小总代价，检验免费终点规则对路径累计与启发函数的影响。

设计：沿用A*的坐标状态、优先队列和前驱，松弛式改为new_g=g+cost(next)。起点g=0，普通格按地形权值收费，E的进入费用为0。以m为普通格最小费用、cE为终点费用，非终点取h=max(D-1,0)×m+cE，终点取0；至少D-1次普通格进入与最后一次终点进入构成该下界。相邻非终点的h差至多m，直接进入终点时h=cE，均不超过边费用。

设置：PDF的5×5加权地图，S=(0,0)、E=(4,4)，地形费用1、3、8，E费用0。比较h=0、原始曼哈顿D、修正max(D-1,0)，地图、费用、邻居顺序与同分规则相同。

结果：三种设置的最小总代价均为7，无解接口返回-1。路径共移动8次，最后一步免费。

h	总代价	扩展节点	入堆次数	耗时/μs
0	7	14	24	26.64
D	7	10	19	34.70
max(D-1,0)	7	10	19	30.00

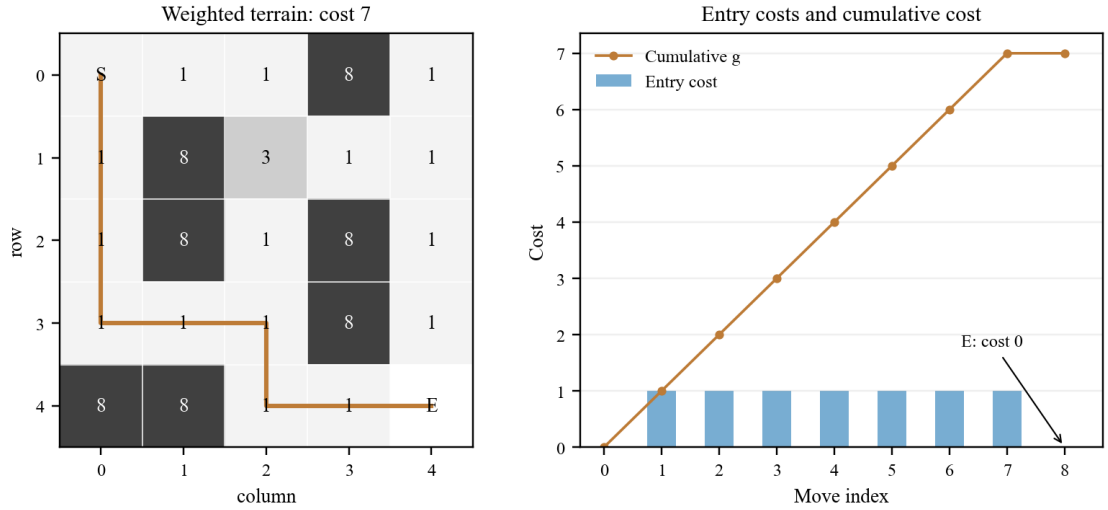


图2 三种h返回相同路径。右图为进入费用与累计g；第8步进入E收费0，累计值保持7。

分析：返回路径为(0,0)→(1,0)→(2,0)→(3,0)→(3,1)→(3,2)→(4,2)→(4,3)→(4,4)，费用序列1、1、1、1、1、1、1、0，累计g为1、2、3、4、5、6、7、7。另一条同为8步的上侧路线经(1,2)的费用3，总代价9，因此本题必须优化权值总和。

分析：原始D与修正h均扩展10次、入堆19次，耗时为34.70、30.00μs。本图非终点的原始h、f比修正值大1，排序相同；终点h均为0。紧邻E的真实剩余费用为0，原始D=1会高估，修正h=0满足可采纳性。原始D直接计算坐标距离，修正h还扫描最小地形费用并作费用换算。

次序	位置	g	原始h	修正h	原始f	修正f
1	(0,0)	0	8	7	8	7
2	(0,1)	1	7	6	8	7
3	(0,2)	2	6	5	8	7
4	(1,0)	1	7	6	8	7
5	(2,0)	2	6	5	8	7
6	(3,0)	3	5	4	8	7
7	(3,1)	4	4	3	8	7
8	(3,2)	5	3	2	8	7
9	(4,2)	6	2	1	8	7
10	(4,3)	7	1	0	8	7
11	(4,4)	7	0	0	7	7

分析：两种启发的11次有效出堆位置依次相同，含终点1次，最终路径含9格，另探索了(0,1)、(0,2)。从(0,2)进入费用3的(1,2)，原始/修正f为10/9；下侧候选为8/7，因此两种设置都转向低费用通道。h=0扩展14次，两种距离启发均减少28.6%。

总结：进入格计费与免费E均已纳入g和h，A*得到代价7；本题的最优路径由累计费用决定，而非单看移动次数。

3. 实验三 极大极小博弈树

目标：求根MAX玩家的最优得分，并比较完整Minimax与Alpha-beta在两种访问顺序下的开销。

设计：叶子返回value，内部节点递归交替执行MAX取最大值、MIN取最小值。Alpha-beta初始α为负无穷、β为正无穷；MAX更新α，MIN更新β，当α≥β时跳过未访问子节点。正负无穷作为初值，使负叶子分数也按相同规则处理。

设置：采用PDF博弈树，根为MAX，三层边展开后有15个节点、8个叶子。基线从左到右，再将各层子节点反序；两种顺序分别运行Minimax与Alpha-beta，共4组。

结果：四组根节点值均为5。完整Minimax访问15个节点、评估8个叶子；Alpha-beta访问14个节点、评估7个叶子。

算法	顺序	访问节点	叶子评估	剪枝事件	耗时/μs
Minimax	左到右	15	8	0	10.95
Minimax	右到左	15	8	0	14.14
Alpha-beta	左到右	14	7	1	10.10
Alpha-beta	右到左	14	7	1	14.94

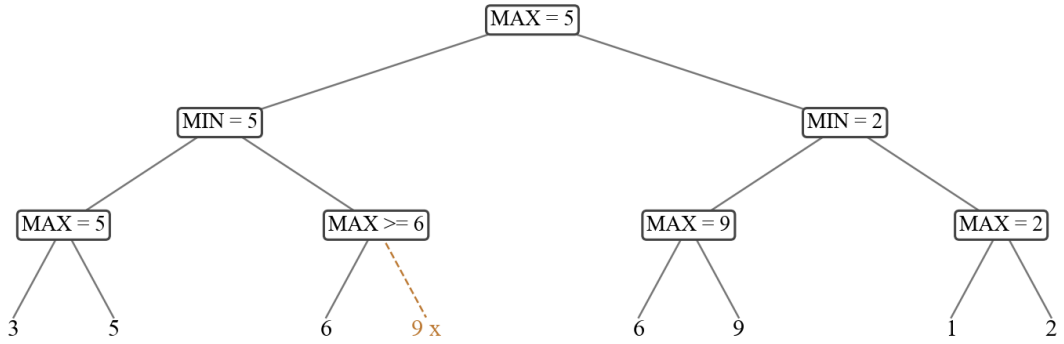


图3 从左到右的数值回传与剪枝。虚线叶子9未访问，所在MAX分支已得到下界6。

分析：底层四个MAX值为5、9、9、2，中层MIN回传min(5,9)=5与min(9,2)=2，根返回max(5,2)=5。左侧MIN先获得5并令β=5，第二个MAX读到6后α=6≥β，后面的9已无法改变该MIN选择5的结果，因此剪去。

分析：反序先得到右侧MIN的界限2，另一MAX读到9后剪去叶子6。两种顺序剪枝位置不同，各省去1个叶子，实测耗时为10.10、14.94μs。

总结：Alpha-beta以一次有效界限判断省去1次叶子评估，减少12.5%的叶子计算，根节点的Minimax结果保持为5。

4. 实验四 竞争环境下的宝藏分配

目标：将最小代价寻路与对抗搜索结合，求A在H=2、H=4时的最优首步和最优收益差。

设计：s记录双方位置、剩余宝藏、双方累计收益与当前玩家；取T后更新并换人。A*预计算距离d，U=Score_A-Score_B；F_T为取T后的递归回传值，达到深度或宝藏收完时返回U；V(T)为宝藏价值，p为当前位置。

设置：PDF的7×7地图，T1至T4价值为9、6、7、7，A/B/T格费用1。经过宝藏不自动收集，另一玩家不阻路，已取位置可通行，距离矩阵全程复用。H按单次行动计数；两算法比较自然、反序、收益排序，共12组。

结果：两算法、三种顺序均得到H=2首选T2、U=-1；H=4首选T3、U=3。路线与A*距离如下。

从/到	A	B	T1	T2	T3	T4
A	0	12	10	4	8	8
B	12	0	8	8	4	10
T1	10	8	0	6	6	12
T2	4	8	6	0	6	6
T3	8	4	6	6	0	6
T4	8	10	12	6	6	0

矩阵中A、B表示初始位置，T表示宝藏位置；对角为0。六个端点费用均为1，反向路线代价相同，因此本题矩阵对称。A→T1的曼哈顿距离为6，A*实际代价为10，Gain采用后者。

例如A首取T2后，状态为(T2,B,{T1,T3,T4},2,0,B回合)，U=2；B随后取T3后，状态变为(T2,T3,{T1,T4},2,3,A回合)，U=-1，达到H=2的评价深度。

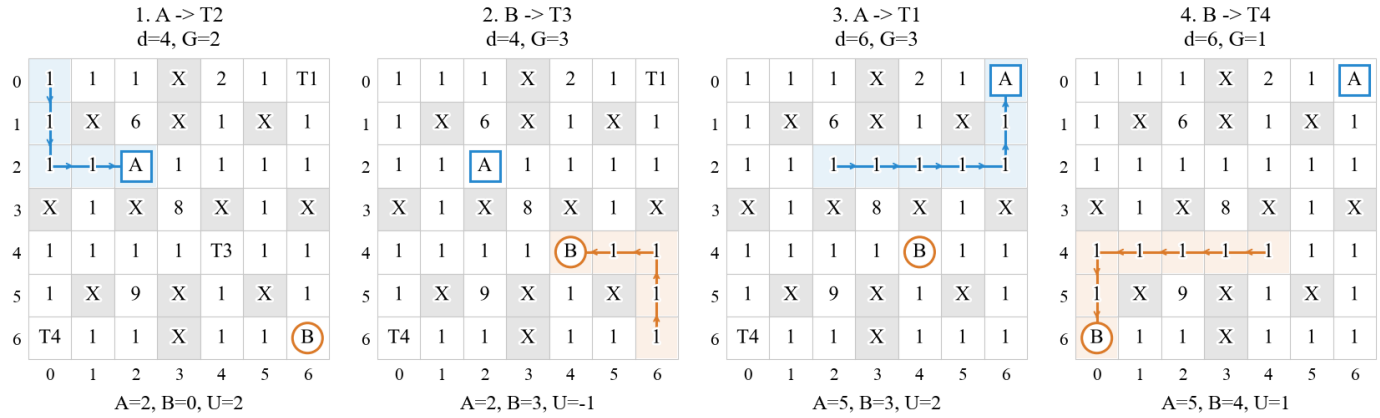


图4 H=2首步的四回合展开：先由H=2选T2，后续完整搜索剩余回合。第二步U=-1为原深度评价，四步累计U=1。

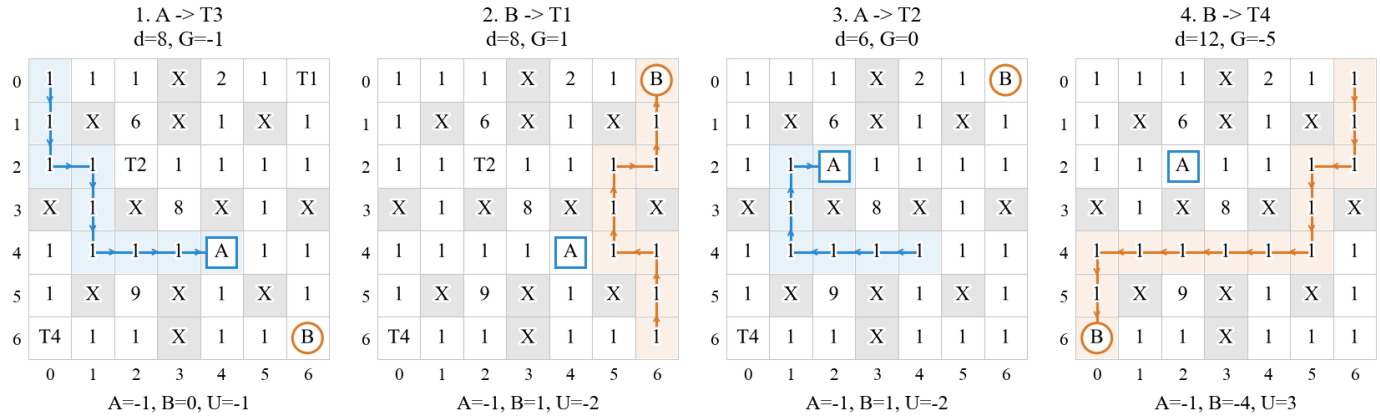


图5 H=4四回合路线。两图同宽、同一地图比例；方形A、圆形B，d为路费，G为本次收益，U为累计收益差。

分析：H=2只比较前两步，T2分支U=2-3=-1，优于T3的-2，故选T2。图4补齐后A走T2→T1得2+3=5，B走T3→T4得3+1=4，终局U=1。H=4计入后续位置与分配，图5中A得-1+0=-1、B得1-5=-4，终局U=3，高于T2分支的1，故首选T3。

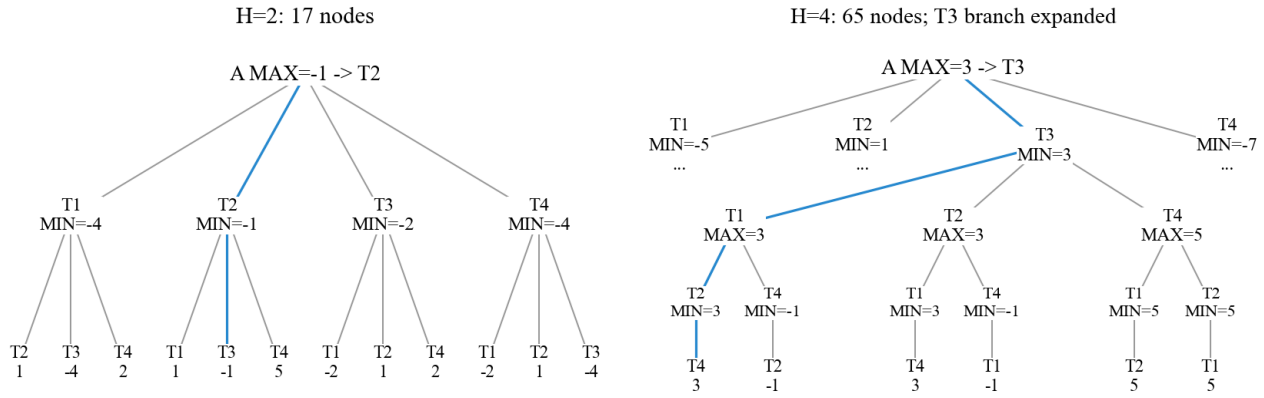


图6 选择树对照。H=2展开至两步并回传当前U；H=4展开最优首步T3下的全部后续分支，其他首步标注回传值。蓝线标示自然顺序保留的最优行动；H=2完整树17个节点，H=4完整树65个节点。

算法与顺序	计算公式与初始顺序	H2节点	H2耗时/μs	H4节点	H4耗时/μs	最优收益差
Minimax 自然	$F_A=\max F_T$，$F_B=\min F_T$ $T^*=\arg\max F_T$；T1→T2→T3→T4	17	30.03	65	122.30	H=2: -1 H=4: 3
Alpha-beta 自然	A:α←max(α,F_T)，B:β←min(β,F_T) α≥β剪枝；T1→T2→T3→T4	13	24.98	47	116.25	H=2: -1 H=4: 3
Alpha-beta 反序	A:α←max(α,F_T)，B:β←min(β,F_T) α≥β剪枝；T4→T3→T2→T1	16	33.41	46	131.00	H=2: -1 H=4: 3
Alpha-beta 收益排序	G(s,T)=V(T)-d(p,T)，G降序；α≥β剪枝 初始T2→T1→T3→T4，各局面重算G	11	28.54	37	77.51	H=2: -1 H=4: 3

分析：G=V-d直接影响当前得分，故先搜G大的动作，期望A更早抬高α、B更早压低β。仅当α≥β、余下分支不能改善祖先选择时剪枝。H2访问17→11；H4自然顺序47→收益排序37，减少21.3%，最优值保持一致。

总结：先搜即时收益最大的T2，H4最终仍选U=3的T3；排序提高剪枝效率，首步由完整回传确定。

5. 自设对照实验与拓展

5.1 跨回合树复用

目标：扩展原有记忆化缓存，沿用动态规划复用子问题结果的思想，减少跨回合重复搜索。

设计：原缓存每次搜索重建，现由会话跨回合保留。键含位置、余宝、玩家、深度，保存F-U及精确值/上下界。

设置：6张10×10八宝藏图（种子20261111至20261116），每图独立会话，换图清空缓存；回放相同8局面，含冷启动。预热3次、测21次；64/256/1024节点预算含中断迭代。

结果：72组完整搜索与108组限额配置。完整搜索与独立穷举一致；表列6图平均8回合总量。

搜索设置	重建节点	复用节点	重建/ms	复用/ms
Alpha-beta+提示，完整深度	1999.0	1174.5	6.923	4.127
Alpha-beta+提示，滚动H4	564.3	527.5	1.795	1.764

分析：同图前回合已搜索的后续局面，可在下一回合匹配缓存后读取。全深度访问1999→1174.5，减少41.2%；1024节点最优选择47/48→48/48。滚动H4与原H3子树深度不同，数值缓存命中较少。

总结：64节点时最优选择43/48→41/48；双步加深恢复43/48，但1024节点仅47/48，故保留逐层复用。

5.2 惯性跳棋策略

目标：比较惯性跳棋策略与A*的路线与开销：A*选定一步后，沿原方向前进一格，再评估和回退。

设计：保存p，沿原方向先进入q再算f=g+h。f(q)≤f(p)继续，f(q)>f(p)退回并恢复A*。保留全部侧枝，保存栈顶与OPEN按(f,h)比较；遇墙、边界或更优g恢复选点。

设置：13张专设/课程图及40张40×40随机图（单位权、加权各20张，种子20261201至20261220），共106配置。对照原A*；预热3次，21批各10次，计入h构造、保存、恢复和路径还原，计时关闭轨迹记录。

结果：两法均53/53最优。表中为A*→惯性跳棋策略；随机20图、直廊2图取均值，优者按耗时判断。

输入	h计算次数	回退数	耗时/μs	优者
课程迷宫	12→14	1	17.5→35.8	A*
课程加权	20→18	3	25.1→43.0	A*
环形捷径	9→8	0	15.3→21.2	A*
加权环形	15→14	0	24.4→36.6	A*
单支死路	9→8	0	14.8→21.9	A*
横/竖直廊	402→401	0	632.5→958.0	A*
蛇形走廊	882→881	390	1414.7→3264.5	A*
开放地图	160→82	0	438.4→413.5	惯性跳棋策略
费用突增	11→9	4	16.2→30.3	A*
强制绕行	12→11	2	19.0→41.0	A*
梳形地图	40→30	0	67.5→84.3	A*
阶梯走廊	42→41	0	87.5→126.9	A*
随机单位权	423.25→367.35	4.8	723.5→1101.2	A*
随机加权	941.15→989.5	187.15	1610.6→3203.5	A*

分析：一致h下沿边f不降，惯性跳棋策略接受f不变的段。开放图延后侧枝，h计算160→82；直廊堆取点401→2，但仍逐格算h并保存399个状态。原A*起点h调用两次。蛇廊绕行使f增大，触发390次回退。

总结：惯性跳棋策略在开放图更快，随机单位权胜3/20、加权胜0/20；路径费与往返费分记。

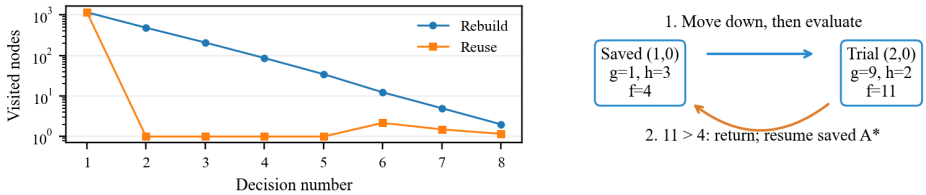


图7 左：跨回合复用与重建的节点访问量；右：惯性跳棋策略前进后算得f由4增至11，回退并恢复A*。